Nama : Mohamad Zaky Bahtiar Arifianto
Matkul : Praktikum Pemrogramman Berorientasi Obyek
Jurusan : Teknik Informatika
NRP : 3125522015

Tugas Pertemuan 10

BAB I: TUJUAN PRAKTIKUM

Praktikum Pemrograman Berorientasi Obyek Modul 10 ini menitikberatkan pada perancangan arsitektur perangkat lunak yang tangguh (robust) terhadap anomali masukan dan kegagalan sistem. Tujuan spesifik dari praktikum ini meliputi:

- **Membedakan Validasi Input dan Penanganan Exception:** Memahami secara konseptual bahwa validasi masukan berfungsi mendeteksi kondisi yang tidak diperbolehkan pada batas terluar sistem, sedangkan exception digunakan sebagai sinyal struktural untuk menyampaikan kegagalan logika ke pemanggil tanpa merusak alur eksekusi utama.
- **Menguasai Mekanisme try, catch, finally, throw, dan throws:** Mampu menempatkan blok penanganan secara modular: melempar eksepsi (throw/throws) pada lapisan Model dan menangkapnya (try-catch) pada JavaFX Controller untuk menampilkan pesan kesalahan yang ramah pengguna.
- **Membuat dan Mengintegrasikan Custom Exception Domain MBG:** Mengembangkan checked exception khusus (StokTidakCukupException) yang merepresentasikan kegagalan transaksi pengeluaran logistik bahan baku secara presisi.
- **Memastikan Integritas dan Konsistensi State Objek (State Invariance):** Membuktikan bahwa setiap operasi yang ditolak oleh sistem wajib mempertahankan nilai atribut internal objek persis sama seperti sebelum operasi dipanggil, mencegah timbulnya efek samping (side effects) atau data korup.
- **Menjaga Stabilitas Aplikasi Desktop Enterprise:** Mencegah terjadinya crash (abnormal termination) pada antarmuka JavaFX dengan menangkap NumberFormatException, IllegalArgumentException, dan StokTidakCukupException melalui kotak dialog Alert.

BAB II: DASAR TEORI & IMPLEMENTASI EXCEPTION HANDLING

Exception merupakan suatu kondisi abnormal yang mengganggu jalannya eksekusi normal program. Tanpa mekanisme penanganan yang terstruktur, terjadinya eksepsi pada thread JavaFX Application Thread akan menyebabkan antarmuka membeku (freeze) atau aplikasi berhenti seketika.

2.1 Hierarki dan Komponen Exception Handling

- **try:** Menandai blok kode berisiko yang berpotensi menghasilkan kesalahan, seperti operasi konversi teks input (Integer.parseInt) dan pemanggilan logika bisnis pengeluaran stok (kurangiStok).
- **catch:** Menangani jenis exception tertentu secara berjenjang dari subkelas yang lebih spesifik hingga kelas yang lebih umum, mencegah aplikasi crash dan mengemas pesan teknis menjadi pesan informatif bagi pengguna.
- **finally:** Menjamin eksekusi langkah akhir pembersihan atau verifikasi konsistensi state objek, baik ketika blok try berhasil maupun saat terjadi kegagalan/eksepsi.

- **throw & throws:** Kata kunci throw digunakan untuk melempar instance objek eksepsi secara eksplisit ketika terjadi pelanggaran prakondisi di dalam method model. Kata kunci throws digunakan dalam deklarasi signature method untuk menginformasikan bahwa method tersebut berpotensi meneruskan checked exception ke pemanggilnya.

2.2 Analisis Penanganan Input Tidak Valid (Kasus 'abc')

Ketika pengguna memasukkan string huruf seperti 'abc' pada field jumlah pemakaian stok, metode Integer.parseInt(rawInput) di dalam Controller akan secara otomatis melempar NumberFormatException. Sistem menangani kesalahan ini secara presisi pada blok catch teratas:

[Kode Arsitektur MVC MainController.java]:
```
try {
    int jumlah = Integer.parseInt(rawInput);
    dipilih.kurangiStok(jumlah);
    repo.simpanAtauPerbaruiBahanBaku(dipilih);
    tampilkanAlert(Alert.AlertType.INFORMATION, 'Pengeluaran Stok Berhasil', ...);
} catch (NumberFormatException ex) {
    tampilkanAlert(Alert.AlertType.ERROR, 'Format Input Salah (NumberFormatException)',
            'Ditolak: input harus bilangan bulat.',
            'Input yang dimasukkan: "' + rawInput + '". State stok objek tetap utuh.');
} catch (IllegalArgumentException ex) {
    tampilkanAlert(Alert.AlertType.WARNING, 'Input Tidak Valid', 'Ditolak: ' + ex.getMessage(), ...);
} catch (StokTidakCukupException ex) {
    tampilkanAlert(Alert.AlertType.ERROR, 'Stok Gudang Tidak Cukup', 'Ditolak: ' + ex.getMessage(), ...);
} finally {
    tblBahan.refresh();
    lblStateAkhir.setText('Stok akhir: ' + dipilih.getStok() + ' (Terverifikasi di finally)');
}
```

BAB III: VALIDASI STATE OBJEK & LOGIKA BISNIS MODEL

Prinsip fundamental state validation pada Pemrograman Berorientasi Obyek menegaskan bahwa objek bertanggung jawab penuh atas keabsahan datanya sendiri (encapsulation & information hiding).

3.1 Logika Bisnis Model BahanBaku (BahanBaku.java)

Di dalam kelas BahanBaku, aturan bisnis pengeluaran stok diimplementasikan dengan memeriksa seluruh prakondisi sebelum state internal (atribut stok) dimutasi:

- **Pemeriksaan Bilangan Non-Positif (<= 0):** Jika input pemakaian bernilai nol atau negatif (misal: '0' atau '-1'), metode segera melempar IllegalArgumentException dengan pesan 'Jumlah pemakaian harus lebih dari nol.' Operasi langsung dihentikan sebelum atribut stok berkurang.
- **Pemeriksaan Ketersediaan Stok Fisik (jumlah > stok):** Jika permintaan melebihi stok yang ada (misal: stok tersedia 3 kg, tetapi diminta 9 kg), metode melempar custom checked exception StokTidakCukupException. Nilai stok tidak dipotong sedikitpun dan tetap bernilai 3 kg.

- **Mutasi State yang Aman (stok -= jumlah):** Mutasi pengurangan state internal hanya dieksekusi setelah seluruh pemeriksaan prakondisi di atas terpenuhi 100%. Hal ini menjamin bahwa objek tidak pernah berada dalam keadaan invalid (misal: stok bernilai negatif).

3.2 Validasi Atomic State pada Menu & Komposisi Gizi (Menu.java)

Pada modul Menu & Gizi, relasi Aggregation antara Menu dan BahanBaku menerapkan validasi atomic state. Sebelum memproduksi porsi menu, sistem memeriksa masa kedaluwarsa dan kecukupan stok seluruh komoditas bahan baku dalam komposisi. Jika salah satu bahan habis atau kedaluwarsa, seluruh transaksi dibatalkan total sehingga tidak terjadi pengurangan stok parsial.

BAB IV: ANALISIS HASIL PENGUJIAN & TERMINAL LOG

Pengujian dilakukan secara berurutan terhadap objek tunggal BahanBaku (komoditas Dada Ayam Fillet Segar) dengan stok awal = 5 unit (kg). Hasil pengujian terminal dievaluasi untuk membuktikan konsistensi state.

Uji	Input	Hasil yang Diharapkan	Hasil Aktual Sistem	State Akhir (finally)
1	**"2"**	Pemakaian berhasil	Pemakaian stok berhasil.	**3 kg**
2	**"0"**	Ditolak: jumlah tidak positif	Ditolak (Jumlah pemakaian harus lebih dari nol.)	**3 kg**
3	**"9"**	Ditolak: stok tidak mencukupi	Ditolak (Stok tidak cukup untuk Dada Ayam Fillet Segar. Tersedia: 3 kg, diminta: 9 kg)	**3 kg**
4	**"abc"**	Ditolak: format angka salah	Ditolak karena format angka salah (input harus bilangan bulat).	**3 kg**
5	**"-1"**	Ditolak: jumlah tidak positif	Ditolak (Jumlah pemakaian harus lebih dari nol.)	**3 kg**
6	**"3"**	Pemakaian berhasil (stok habis)	Pemakaian stok berhasil.	**0 kg**

4.1 Analisis Detail Berdasarkan Skenario Pengujian

- **Skenario 1 (Input '2'):** Masukan numerik valid dan jumlah permintaan (2) lebih kecil dari stok awal (5). Metode kurangiStok berhasil dijalankan sehingga stok tersisa 3 kg.
- **Skenario 2 (Input '0'):** Nilai nol ditolak oleh validasi batas (jumlah <= 0). IllegalArgumentException dilempar dan stok dipertahankan tetap 3 kg.
- **Skenario 3 (Input '9'):** Permintaan sebesar 9 kg melebihi stok yang tersedia (3 kg). Metode melempar StokTidakCukupException. Mutasi state dibatalkan sehingga stok akhir tetap utuh 3 kg.

- **Skenario 4 (Input 'abc'):** String non-numerik gagal dikonversi oleh Integer.parseInt, memicu NumberFormatException pada lapisan Controller. Pemanggilan logika model tidak pernah dilakukan dan state stok tetap aman di 3 kg.
- **Skenario 5 (Input '-1'):** Input bernilai negatif ditolak oleh validasi prakondisi model dengan IllegalArgumentException. Konsistensi state terjaga pada 3 kg.
- **Skenario 6 (Input '3'):** Pembelian tepat sejumlah sisa stok (3 kg dari ketersediaan 3 kg) berhasil dieksekusi secara legal, membuktikan bahwa batas evaluasi (jumlah > stok) bekerja akurat dan menghasilkan stok akhir 0 kg tanpa defisit.

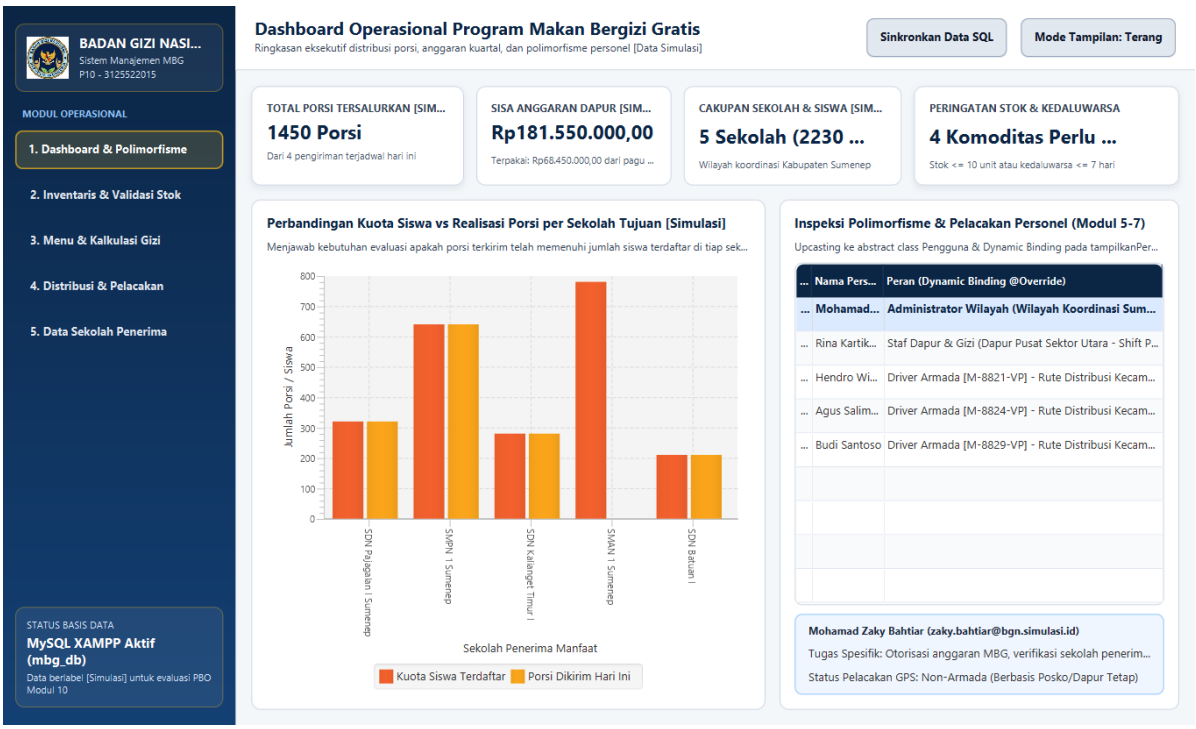
BAB V: KESIMPULAN

Dari perancangan, implementasi, dan pengujian yang telah dilaksanakan pada proyek Sistem Manajemen Program Makan Bergizi Gratis (MBG), dapat disimpulkan bahwa:

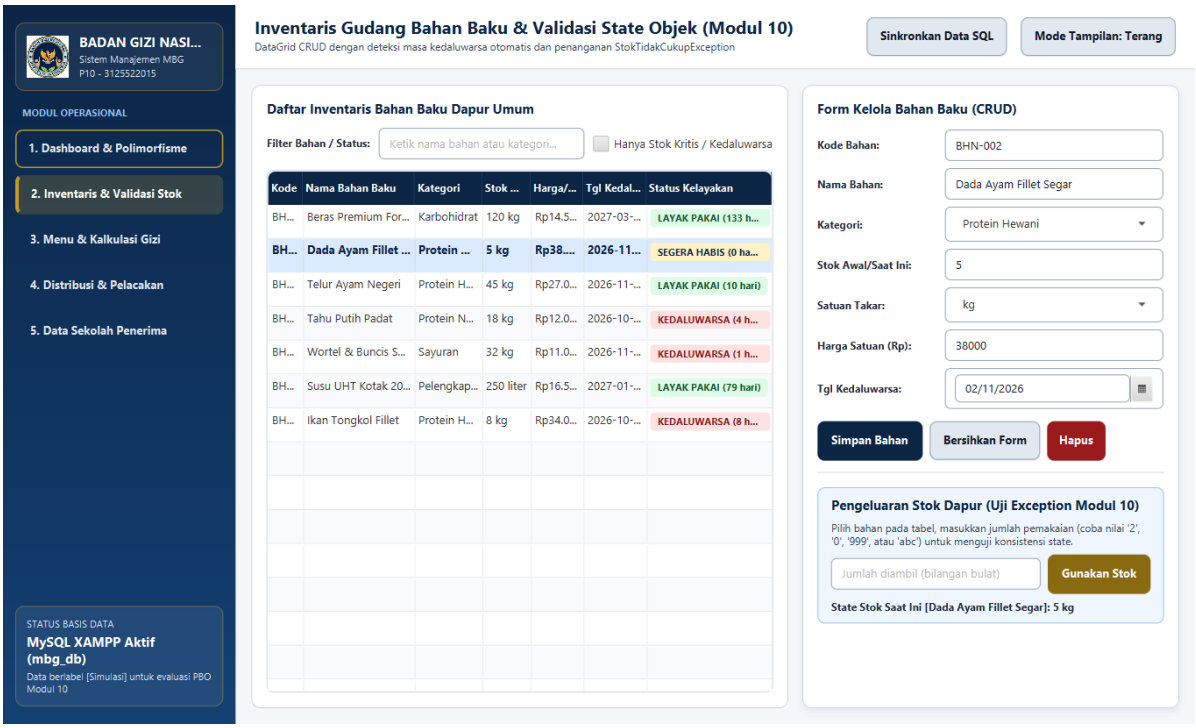
- **Pencegahan Crash Aplikasi Desktop:** Penerapan blok try-catch-finally pada JavaFX Controller terbukti efektif mengisolasi seluruh potensi kegagalan masukan pengguna (seperti kesalahan format teks dan angka di luar batas nalar), mencegah Java Virtual Machine (JVM) menghentikan aplikasi secara tiba-tiba.
- **Jaminan Konsistensi State (State Invariance):** Pemisahan antara tahap validasi prakondisi dan tahap mutasi state menjamin bahwa kegagalan operasi tidak pernah meninggalkan efek samping (side-effects). Nilai stok terbukti 100% konsisten pada setiap pengujian gagal.
- **Keunggulan Custom Checked Exception:** Pembuatan kelas StokTidakCukupException memberikan semantik kesalahan domain yang spesifik dan memaksa arsitektur perangkat lunak untuk menangani skenario kekurangan stok logistik secara terstruktur dan formal.

BAB VI: LAMPIRAN BUKTI IMPLEMENTASI & PENGUJIAN

Berikut adalah dokumentasi visual antarmuka JavaFX dan keluaran terminal IDE yang membuktikan keberhasilan implementasi Modul 10:



Gambar 1: Dashboard Operasional Program MBG (JavaFX) - Menampilkan Polimorfisme Pengguna & Manajemen State



Gambar 2: Antarmuka Inventaris Gudang Bahan Baku - Uji Eksepsi & Deteksi Masa Kedaluwarsa Otomatis



Gambar 3: Log Eksekusi Terminal Antigravity IDE - Verifikasi Exception Handling & Konsistensi State Objek